

Hibernate Framework

exam-oriented explanation with important exam questions

Below is **exam-oriented, crisp, high-scoring** notes on **Hibernate Framework**, along with **important university exam questions (theory + viva)**.

HIBERNATE FRAMEWORK — EXAM ORIENTED NOTES

Hibernate is an **open-source ORM (Object Relational Mapping)** framework in Java that maps **Java objects** to **relational database tables**.

1. What is Hibernate?

Hibernate is a **persistence framework** that automates the mapping between Java classes and database tables.

It removes the need for writing **JDBC boilerplate code**, improves productivity, and supports advanced ORM features.

Key Features

- ORM (Object Relational Mapping)
 - Automatic table creation (DDL)
 - Caching mechanism
 - HQL (Hibernate Query Language)
 - Lazy loading & Eager loading
 - Transaction management
 - Supports multiple databases
-

2. ORM (Object Relational Mapping)

ORM is the technique of mapping **Java objects** → **database tables** and **object attributes** → **table columns**.

Advantages of ORM

- No need to write SQL repeatedly

- Clean, object-oriented persistence
 - DB-independent
 - Faster development
 - Automatic management of relationships (One-to-Many, Many-to-Many)
-

3. Hibernate Architecture

Very important for exams — draw diagram.

Components

1. Configuration
2. SessionFactory
3. Session
4. Transaction
5. Query / HQL
6. ORM Mapping files (XML / Annotations)

Explanation

- **Configuration:** Loads `hibernate.cfg.xml`
 - **SessionFactory:** Creates database connections (heavy object; created once)
 - **Session:** Lightweight object used for CRUD
 - **Transaction:** Ensures ACID operations
 - **Query:** Used to execute HQL/SQL
-

4. Hibernate Mapping

Mapping means telling Hibernate which Java class corresponds to which database table.

Ways of Mapping

1. XML Mapping (`*.hbm.xml`)
2. Annotation-based mapping
 - `@Entity`
 - `@Table`
 - `@Id` , `@Column`
 - `@GeneratedValue`

Example

java

 Copy code

```
@Entity
@Table(name="student")
public class Student {

    @Id
    @GeneratedValue
    private int id;

    private String name;
}
```

5. Hibernate Configuration

Main configuration file: `hibernate.cfg.xml`

Contains:

- Database connection properties
- Hibernate dialect
- Mapping resources

Example:

xml

 Copy code

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost/test</property>
    <mapping class="com.model.Student"/>
  </session-factory>
</hibernate-configuration>
```

6. CRUD Operations in Hibernate

CRUD means Create, Read, Update, Delete using `Session` .

Operation	Hibernate Method
Create	save() / persist()
Read	get() / load()
Update	update()
Delete	delete()

Example:

java

 Copy code

```
session.beginTransaction();
session.save(student);
session.getTransaction().commit();
```

7. Hibernate HQL (Hibernate Query Language)

HQL = object-oriented SQL-like language.

Example:

java

 Copy code

```
Query q = session.createQuery("from Student");
List<Student> list = q.list();
```

8. Hibernate Inheritance Mapping

Important theory question.

Types:

1. Single Table Strategy (@Inheritance(SINGLE_TABLE))
2. Table Per Class
3. Joined Subclass Strategy (normalized)

9. Hibernate Caching

Caching improves performance by reducing database hits.

Levels of Cache

1. **First-Level Cache** (Session level) – enabled by default
2. **Second-Level Cache** (SessionFactory level) – needs configuration

Examples: EHCACHE, OSCache, Infinispan

10. Hibernate vs JDBC (Exam Favourite Question)

Hibernate	JDBC
No SQL required (HQL)	Manual SQL writing
Caching available	No caching
ORM tool	Not ORM
Automatic table handling	Manual schema handling

11. Hibernate Annotations

Very important for viva.

- @Entity
- @Id
- @GeneratedValue
- @Table(name="")
- @Column(name="")
- @OneToOne , @ManyToOne , @ManyToMany

12. Transactions in Hibernate

Hibernate follows ACID properties.

java

 Copy code

```
Transaction t = session.beginTransaction();  
session.save(obj);  
t.commit();
```

13. Hibernate Lifecycle

1. **Transient** (Object created, not associated with session)
2. **Persistent** (Object saved and associated with session)
3. **Detached** (Session closed, object exists)
4. **Removed** (Deleted)

14. Hibernate Relationships

Important for university questions.

- One-to-One
- One-to-Many
- Many-to-One
- Many-to-Many

Example:

```
java
```

 Copy code

```
@OneToMany(mappedBy="dept")  
private List<Employee> employees;
```

IMPORTANT EXAM QUESTIONS (THEORY + DESCRIPTIVE)

Short Notes

1. Hibernate Architecture
2. HQL & Criteria API
3. Hibernate Caching
4. SessionFactory
5. ORM Concept

6. Collection mapping
7. Hibernate vs JDBC
8. Hibernate annotations

Long Questions

1. Explain Hibernate Architecture with neat diagram.
2. What is ORM? Explain advantages of ORM.
3. Explain different object states in Hibernate lifecycle.
4. Explain HQL with examples.
5. Describe inheritance mapping strategies.
6. Explain First-Level and Second-Level caching.
7. Explain the difference between `get()` and `load()` .
8. Explain steps to create a Hibernate application.
9. Explain Hibernate mapping using XML and annotations.
10. What are relationships in Hibernate? Describe all types.



VIVA QUESTIONS (Very Common)

1. What is Hibernate?
2. What is ORM?
3. What is Session and SessionFactory?
4. What is the difference between `save()` and `persist()` ?
5. What is lazy loading?
6. What is caching in Hibernate?
7. Difference between `get()` and `load()`?
8. What is HQL?
9. What is the default cache in Hibernate?
10. How does Hibernate manage transactions?

If you want, I can also prepare:

- ✓ Handwritten-style PDF notes
- ✓ Diagram of Hibernate Architecture
- ✓ Unit-wise exam notes
- ✓ 2-marker / 5-marker / 10-marker answers

Just tell me!



Introduction to Hibernate Framework o Architecture

Below is a **clear, exam-oriented explanation** of **Introduction to the Hibernate Framework** and **Hibernate Architecture** — perfect for 5-mark or 10-mark answers.

✅ INTRODUCTION TO HIBERNATE FRAMEWORK — EXAM ORIENTED

Hibernate is a **Java-based ORM (Object Relational Mapping)** framework used to interact with relational databases in an object-oriented manner.

Instead of writing lengthy JDBC code, Hibernate allows developers to **save, update, delete, and fetch Java objects directly to/from a database table**.

Why Hibernate?

Hibernate solves the mismatch between:

- **Object-Oriented Java classes**
- **Relational Database tables**

This problem is called the **Object-Relational Impedance Mismatch**, and Hibernate removes it by automatically mapping **Java objects → tables and fields → columns**.

Key Features

- Eliminates most SQL writing
- Automatic table creation (DDL)
- HQL (Hibernate Query Language)
- Caching for performance
- Connection pooling
- Supports annotations and XML
- Database independent (dialects)

Advantages of Hibernate

- Reduces boilerplate JDBC code

- Faster development
- Portable across databases
- Supports transactions
- Supports lazy loading
- Supports complex associations (One-to-One, Many-to-Many)



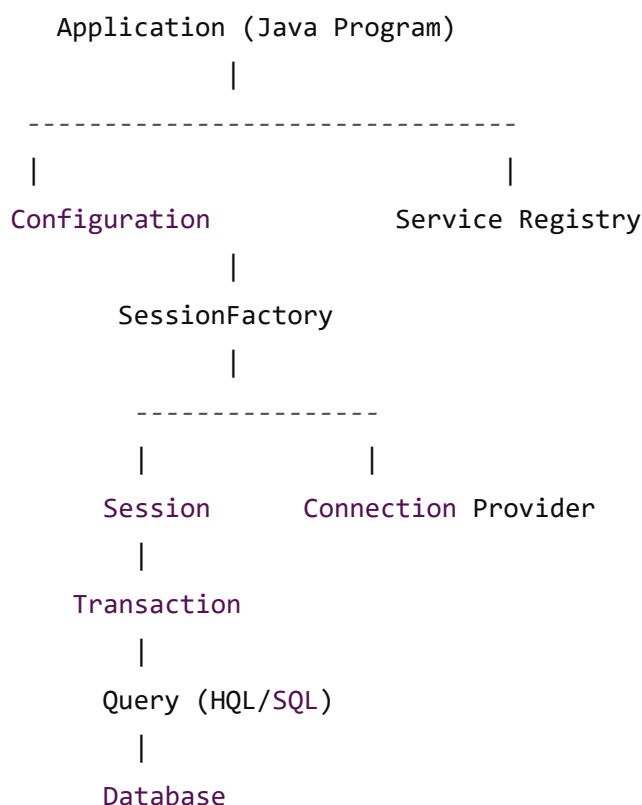
HIBERNATE ARCHITECTURE — EXAM ORIENTED

Hibernate architecture consists of several layers and components that work together to manage database operations.

Diagram (You can draw this in exam)

pgsql

Copy code



COMPONENTS OF HIBERNATE ARCHITECTURE

1. Configuration

- Reads `hibernate.cfg.xml`
- Contains DB settings (url, username, password, dialect)
- Loads mapping files or annotated classes

- First step of Hibernate initialization

java

 Copy code

```
Configuration cfg = new Configuration().configure();
```

2. Service Registry

- A support component used internally
 - Builds SessionFactory
 - Manages services like connection pooling, transaction handling
-

3. SessionFactory

- **Very important in exams**
- Heavy-weight object, created once per application
- Thread-safe
- Used to create **Session objects**

Tasks:

- Holds DB connection details
- Caches metadata
- Provides second-level cache

java

 Copy code

```
SessionFactory factory = cfg.buildSessionFactory();
```

4. Session

- Light-weight object (not thread-safe)
- Represents a single unit of work
- Used for CRUD operations

java

 Copy code

```
Session session = factory.openSession();
```

5. Transaction

- Used to group operations into a single logical unit
- Ensures ACID properties
- Always used with Session

java

 Copy code

```
Transaction tx = session.beginTransaction();
```

6. Query

Used to perform database operations using:

- HQL (Hibernate Query Language – object-oriented)
- Native SQL
- Criteria API

Example:

java

 Copy code

```
Query q = session.createQuery("from Student");
```

7. ORM Mapping Layer

Defines how Java classes are mapped to table structures.

Two ways:

- XML Mapping (*.hbm.xml)
 - Annotations (@Entity, @Id, @Table, @Column)
-

8. Database

Hibernate interacts with the relational database using JDBC internally.

But developers don't need to write JDBC code manually.



EXAM ANSWER READY SUMMARY (Write this in exams)

Hibernate is an ORM framework that maps Java objects to relational database tables. The architecture of Hibernate consists of components such as Configuration, Service Registry, SessionFactory, Session, Transaction, Query, and the database. The application interacts with Hibernate through the Session object, which performs CRUD operations. SessionFactory is a heavy-weight object responsible for creating sessions and managing second-level cache. Hibernate internally uses JDBC to communicate with the database but hides the complexity from developers.



IMPORTANT EXAM QUESTIONS

1. Explain Hibernate Framework.
2. Explain Hibernate Architecture with a neat diagram.
3. What is SessionFactory? Why is it important?
4. Difference between Session and SessionFactory.
5. Explain the role of Configuration and Query interface.

If you want, I can prepare **Unit-wise Notes**, **Diagram PDF**, or **10-mark long answers**.



Introduction to Hibernate Framework
o Architecture
□ HibernateinIDE
o Creating web application using Hibernate API exam-oriented explanation with important exam questions

Below is a **complete, exam-oriented explanation** for:

- ✓ Introduction to Hibernate Framework
- ✓ Hibernate Architecture
- ✓ Hibernate in IDE (Using Eclipse/NetBeans/IntelliJ)
- ✓ Creating a Web Application using Hibernate API

↑ Each topic is explained in **easy language**, perfect for **5-marks / 10-marks answers** with **important exam questions**.

★ 1. INTRODUCTION TO HIBERNATE FRAMEWORK (Exam-Oriented)

Hibernate is an **open-source ORM (Object Relational Mapping)** framework in Java. It allows developers to **store, retrieve, update, and delete Java objects directly in a relational database** without writing long JDBC code.

Why Hibernate? (5 Marks)

- Eliminates repetitive JDBC code
- Supports object-oriented query language — **HQL**
- Database independent (via Hibernate Dialects)
- Automatic table creation
- Handles relationships (1–1, 1–M, M–M)
- Provides caching → improves performance

Hibernate ORM Concept

It maps:

- Java Class → Database Table
- Class Field → Table Column
- Object → Row

★ 2. HIBERNATE ARCHITECTURE (10 Marks)

A common exam question: "Explain Hibernate Architecture with diagram"

Diagram (Write in exam)

pgsql

Copy code

Application Layer

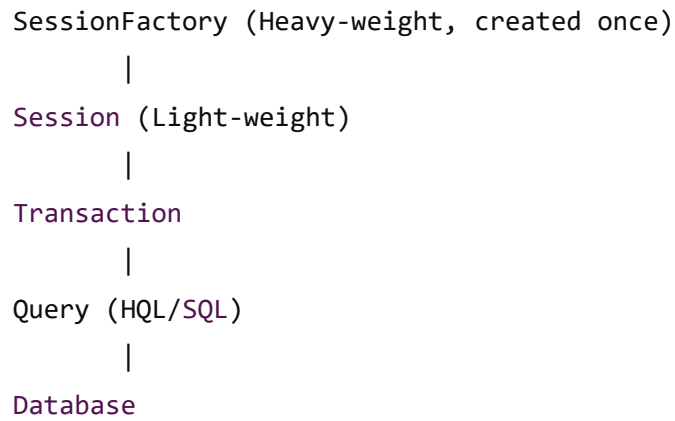
|

Configuration ---- Mapping Files / Annotations

|

Service Registry

|



COMPONENTS

1. Configuration

- Reads `hibernate.cfg.xml`
- Loads DB connection details
- Loads mapping resources

2. Service Registry

- Internal service manager
- Helps build SessionFactory

3. SessionFactory

- Heavy object, created once
- Thread-safe
- Creates **Session** objects
- Stores second-level cache

4. Session

- Used for CRUD operations
- Not thread-safe

5. Transaction

- Manages ACID properties

6. Query

- Used to execute **HQL**, **Criteria**, or **SQL**

7. Database Layer

- Actual relational DB (MySQL, Oracle, PostgreSQL...)

- Hibernate uses JDBC internally to communicate

★ 3. HIBERNATE IN IDE (Exam-Oriented Explanation)

Most exams ask: "Explain how to use Hibernate in an IDE."

Steps to Use Hibernate in Eclipse/NetBeans/IntelliJ

Step 1: Create a Java or Web Project

- File → New → Dynamic Web Project / Maven Project

Step 2: Add Hibernate Libraries

Two ways:

1. Add **Hibernate JAR files** manually
2. Or use **Maven dependency**:

xml

 Copy code

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>
```

Step 3: Add MySQL Connector

xml

 Copy code

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.30</version>
</dependency>
```

Step 4: Create Hibernate Configuration File

hibernate.cfg.xml

Contains:

- Database URL
- Username, password

- Dialect
- Mapped classes

Step 5: Create Entity Class

Use annotations like:

java

 Copy code

```
@Entity
@Table(name="student")
public class Student {
    @Id
    @GeneratedValue
    private int id;
    private String name;
}
```

Step 6: Write Hibernate Utility Class / Test Class

java

 Copy code

```
SessionFactory factory = new Configuration().configure().buildSessionFactory();
Session session = factory.openSession();
```

★ 4. CREATING A WEB APPLICATION USING HIBERNATE API (10 Marks)

This is a frequent long-answer question in exams.

Steps to Create a Hibernate-Based Web Application

Step 1: Create Dynamic Web Project

- File → New → Dynamic Web Project
- Add Apache Tomcat as server

Step 2: Add Hibernate + MySQL JARs

OR

Use Maven dependencies:

xml

 Copy code

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>

<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.30</version>
</dependency>
```

Step 3: Create Database

Example:

sql

 Copy code

```
CREATE DATABASE college;
```

Step 4: Create hibernate.cfg.xml File

Place it in src/main/resources .

xml

 Copy code

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/college</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <mapping class="com.model.Student"/>
  </session-factory>
</hibernate-configuration>
```

```
</session-factory>
</hibernate-configuration>
```

Step 5: Create Entity Class

java

 Copy code

```
@Entity
@Table(name="student")
public class Student {
    @Id @GeneratedValue
    private int id;

    private String name;
    private String email;
}
```

Step 6: Create DAO Class (Data Access Layer)

java

 Copy code

```
public class StudentDAO {

    public void saveStudent(Student s) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();
        session.save(s);
        tx.commit();
        session.close();
    }
}
```

Step 7: Create Servlet for User Input

java

 Copy code

```
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        Student s = new Student();
        s.setName(req.getParameter("name"));
        s.setEmail(req.getParameter("email"));

        new StudentDAO().saveStudent(s);
        resp.getWriter().print("Success!");
    }
}
```

Step 8: Create JSP Page for Form

html

 Copy code

```
<form action="register" method="post">
    Name: <input type="text" name="name" /><br>
    Email: <input type="text" name="email" /><br>
    <input type="submit" value="Register"/>
</form>
```

IMPORTANT EXAM QUESTIONS

5-Marks Questions

1. What is Hibernate? Explain features.
2. Explain steps to use Hibernate in an IDE.
3. What is Hibernate configuration file?
4. Define SessionFactory and Session.
5. Write note on HQL.

10-Marks Questions

1. Explain Hibernate Architecture with a neat diagram.
2. Explain the steps to create a Web Application using Hibernate.
3. Explain XML vs Annotation-based mapping in Hibernate.

4. Explain Hibernate lifecycle states (Transient, Persistent, Detached).
5. Explain the working of Hibernate with example program.

If you want, I can create a **PDF of these notes**, **Handwritten-style notes**, or **full unit notes**.



Hibernate in IDE
o Creating web application using Hibernate API

Below is a **clean, exam-ready explanation** for:

- ✓ **Hibernate in IDE**
- ✓ **Creating Web Application using Hibernate API**
- ✓ **Important exam questions**

Use this directly for **5-marks / 10-marks university answers**.

★ 1. Hibernate in IDE (Exam-Oriented Explanation)

Hibernate in IDE means using Hibernate Framework inside an Integrated Development Environment like **Eclipse, NetBeans, or IntelliJ IDEA** to develop Java or web applications easily.

Why use Hibernate in an IDE?

- Provides automatic code completion
- Easier project setup
- Error checking
- Maven/Gradle support
- Built-in server + DB tools



Steps to Configure Hibernate in IDE (Eclipse Example)

Step 1: Create a New Project

You can choose:

- **Java Project** (Standalone Application)
- **Dynamic Web Project** (Servlet/JSP Application)
- **Maven Project** (Recommended)

Step 2: Add Hibernate and Database Dependencies

Using Maven (Recommended)

Add these dependencies in `pom.xml` :

xml

 Copy code

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>

<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.30</version>
</dependency>
```

OR Add JAR Files Manually

- hibernate-core.jar
- hibernate-commons-annotations.jar
- hibernate-jpa.jar
- mysql-connector.jar

Step 3: Create Hibernate Configuration File

Create `hibernate.cfg.xml` inside `src/main/resources` .

xml

 Copy code

```
<hibernate-configuration>
  <session-factory>
```

```
<property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
<property name="hibernate.connection.url">jdbc:mysql://localhost:3306/test</property>
<property name="hibernate.connection.username">root</property>
<property name="hibernate.hbm2ddl.auto">update</property>
<property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>

<mapping class="com.model.Student"/>
</session-factory>
</hibernate-configuration>
```

Step 4: Create Entity Class

java

 Copy code

```
@Entity
@Table(name="student")
public class Student {
    @Id @GeneratedValue
    private int id;
    private String name;
    private String email;
}
```

Step 5: Create Hibernate Utility Class

java

 Copy code

```
public class HibernateUtil {
    private static SessionFactory factory;

    static {
        factory = new Configuration().configure().buildSessionFactory();
    }

    public static SessionFactory getSessionFactory() {
        return factory;
    }
}
```

✓ This completes Hibernate setup inside IDE.

★ 2. Creating Web Application Using Hibernate API (10 Marks)

This question is **VERY common** in Java EE exams.

Write this answer **stepwise** for full marks.

🔥 Steps to Create a Web Application Using Hibernate

Step 1: Create Dynamic Web Project

- File → New → Dynamic Web Project
- Select Tomcat server

Step 2: Add Required Libraries

Either:

- Add Hibernate + MySQL JARs manually
- OR
- Use Maven dependencies

Step 3: Create Database

Example:

```
sql
```

📋 Copy code

```
CREATE DATABASE college;
```

Step 4: Create `hibernate.cfg.xml`

Place in `src/main/resources` .

Step 5: Create Entity Class (Model Layer)

java

 Copy code

```
@Entity
@Table(name="student")
public class Student {
    @Id @GeneratedValue
    private int id;
    private String name;
    private String email;
}
```

Step 6: Create DAO Class

DAO contains database logic using Hibernate API.

java

 Copy code

```
public class StudentDAO {

    public void save(Student s) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction tx = session.beginTransaction();
        session.save(s);
        tx.commit();
        session.close();
    }
}
```

Step 7: Create a Servlet (Controller Layer)

java

 Copy code

```
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {

    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws ServletException, IOException {

        Student s = new Student();
```



```
s.setName(req.getParameter("name"));
s.setEmail(req.getParameter("email"));

new StudentDAO().save(s);

resp.getWriter().println("Registration Successful!");
}
}
```

Step 8: Create JSP Page (View Layer)

register.jsp

html

 Copy code

```
<form action="register" method="post">
  Name: <input type="text" name="name"/><br>
  Email: <input type="text" name="email"/><br>
  <input type="submit" value="Register"/>
</form>
```

Step 9: Run the Project

- Start Tomcat
- Open browser → <http://localhost:8080/YourProject/register.jsp>
- Submit form
- Hibernate saves data into DB



This completes a full Hibernate Web Application (MVC + Hibernate)



Exam Answer Summary

5 Marks – SHORT

To create a Hibernate-based web application, we need a Dynamic Web Project, Hibernate libraries, a configuration file, entity class, DAO, Servlet, and JSP. Hibernate

API is used in DAO to perform operations like save(), update(), delete(), and load(). The Servlet acts as a controller and JSP acts as the view.

10 Marks – LONG

(Explain with all steps: project creation → config → entity → DAO → servlet → JSP → execution.)

IMPORTANT EXAM QUESTIONS

5 Marks

1. What are the steps to configure Hibernate in IDE?
2. What is the use of `hibernate.cfg.xml` ?
3. What is the role of DAO in Hibernate?
4. Write a note on Hibernate + Web Project setup.

10 Marks

1. Explain how to create a web application using Hibernate API.
 2. Explain Hibernate in IDE with steps.
 3. Explain integration of Hibernate with Servlets/JSP.
 4. Explain Model, View, Controller (MVC) in Hibernate Web App.
-

If you want, I can prepare a **PDF**, **diagram**, or **handwritten-style notes** for these topics.



Lifecycle of Hibernate Entities
□ HB with annotation example

Below is a **perfect exam-oriented answer** for:

- ✓ Lifecycle of Hibernate Entities
- ✓ Hibernate with Annotation – Example Program

Use this directly for 5-marks, 10-marks, or viva.

★ 1. Lifecycle of Hibernate Entities

(VERY IMPORTANT — Frequently asked 5 or 10 marks)

Hibernate entities pass through **four states** during their lifecycle:

✓ 1. Transient State

- The object is created using `new`.
- Not associated with **Session**.
- Not stored in database.
- No persistent identity (primary key is not assigned yet).

Example:

java

 Copy code

```
Student s = new Student();    // Transient  
s.setName("Rahul");
```

✓ 2. Persistent State

- Object becomes persistent when it is **associated with a Hibernate Session**.
- Hibernate tracks changes and automatically updates the database.
- Object has a database identity (primary key).

Example:

java

 Copy code

```
Session session = factory.openSession();  
session.save(s);    // Now object is Persistent
```

✓ 3. Detached State

- When the Session is closed, persistent objects become **detached**.

- Detached objects are not monitored by Hibernate.
- Any changes made will not automatically update the DB.

Example:

java

 Copy code

```
session.close(); // Object becomes Detached
```

✓ 4. Removed State

- The object is scheduled for deletion.
- Hibernate deletes it from DB when transaction is committed.

Example:

java

 Copy code

```
session.delete(s); // Removed state
```

Lifecycle Summary Table (Write in exam)

State	Description	Example
Transient	Not associated with session	<code>new Student()</code>
Persistent	Associated with session, tracked	<code>session.save()</code>
Detached	Session closed	<code>session.close()</code>
Removed	Marked for deletion	<code>session.delete()</code>

This entire lifecycle is a **must-write** answer for 10/10 marks.

★ 2. Hibernate with Annotation – Example Program

This is commonly asked:

“Write Hibernate annotation-based configuration with example”

Below is a **complete** annotation-based Hibernate example:

Step 1: Entity Class with Annotations

java

 Copy code

```
import javax.persistence.*;

@Entity
@Table(name = "student")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "student_name")
    private String name;

    private String email;

    public Student() {}

    // getters and setters
}
```

Explanation of used annotations

Annotation	Meaning
@Entity	Marks class as Hibernate Entity
@Table	Specifies table name
@Id	Marks primary key
@GeneratedValue	Auto PK generation
@Column	Custom column name

Step 2: Hibernate Configuration (hibernate.cfg.xml)

xml

 Copy code

```
<hibernate-configuration>
<session-factory>

<property name="hibernate.connection.url">jdbc:mysql://localhost:3306/college<
<property name="hibernate.connection.username">root</property>
<property name="hibernate.connection.password">root</property>
<property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
<property name="hibernate.hbm2ddl.auto">update</property>

<!-- Annotation Mapping -->
<mapping class="com.model.Student"/>

</session-factory>
</hibernate-configuration>
```

Step 3: Save Entity Using Hibernate API

java

 Copy code

```
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class Main {
    public static void main(String[] args) {

        Configuration cfg = new Configuration().configure();
        SessionFactory factory = cfg.buildSessionFactory();

        Session session = factory.openSession();
        Transaction tx = session.beginTransaction();

        Student s = new Student();
        s.setName("Rahul");
        s.setEmail("rahul@gmail.com");

        session.save(s);    // persistent state
        tx.commit();
```

```
        session.close();    // detached state
        factory.close();
    }
}
```

What to Write in Exam (Short Answer)

In annotation-based Hibernate, Java classes are mapped to database tables using JPA annotations like @Entity, @Table, @Id, and @Column. Hibernate automatically reads these annotations instead of XML mapping files. This reduces configuration and makes mapping clean and easy to maintain.

IMPORTANT EXAM QUESTIONS

- ✓ Explain Hibernate Entity Lifecycle.
 - ✓ Explain Transient, Persistent, Detached and Removed states.
 - ✓ Write a Hibernate program using annotations.
 - ✓ Difference between XML-based and annotation-based mapping.
 - ✓ What is @Entity, @Id, @GeneratedValue annotation?
-

If you want, I can also prepare:

- ✂ Handwritten-style notes PDF
- ✂ Short 5-mark version
- ✂ 10-mark detailed answer

    ...

Hibernate Mappings and Relationships
□ Collection and Component Mapping
□ HQL,Named Queries,Criteria Queries

Below are exam-oriented, point-wise, clear explanations of Hibernate Mappings & Relationships, Collection & Component Mapping, HQL, Named Queries, and Criteria Queries, with important exam questions at the end.

✅ Hibernate Mappings and Relationships

Hibernate supports mapping Java objects (entities) to database tables. Relationships map how entities are associated.

1. Types of Relationships in Hibernate

Hibernate handles the following associations:

a) One-to-One

- One entity is related to exactly one other entity.
- Example: Person ↔ Passport .

Annotation Example:

java

Copy code

```
@OneToOne
@JoinColumn(name="passportId")
private Passport passport;
```

b) One-to-Many

- One parent can have many children.
- Example: Department → Employees .

java

Copy code

```
@OneToMany(mappedBy="department")
private List<Employee> employees;
```

c) Many-to-One

- Many children belong to one parent.
- Example: Many employees belong to one department.

java

```
@ManyToOne
@JoinColumn(name="deptId")
private Department department;
```

d) Many-to-Many

- Many parents to many children.
- Example: Student ↔ Course .

java

 Copy code

```
@ManyToMany
@JoinTable(name="student_course",
    joinColumns=@JoinColumn(name="studentId"),
    inverseJoinColumns=@JoinColumn(name="courseId"))
private List<Course> courses;
```

Cascade Types

Controls what operations flow from parent to children:

- CascadeType.ALL
- PERSIST
- REMOVE
- DETACH
- MERGE
- REFRESH

Fetch Types

Defines how related data is loaded:

- EAGER : Loaded immediately
 - LAZY : Loaded when accessed
-
-

✓ Collection Mapping in Hibernate

Hibernate allows mapping of collections like:

- List
- Set
- Map
- Bag

Used when an entity contains a collection of value types (NOT a separate table entity).

Example: List of phone numbers in a User

java

Copy code

```
@ElementCollection
@CollectionTable(name="user_phones", joinColumns=@JoinColumn(name="userId"))
@Column(name="phone")
private List<String> phones;
```

✓ Component Mapping (Embeddables)

A component (value object) is a class inside another entity — no separate table.

Example: Address component inside Employee

Embeddable Class

java

Copy code

```
@Embeddable
public class Address {
    private String street;
    private String city;
}
```

Entity Class

java

Copy code

```
@Entity
public class Employee {
    @Id
    private int id;
```

```
@Embedded
private Address address;
}
```

- ✓ No separate table
- ✓ Fields of Address stored in Employee table

✓ HQL – Hibernate Query Language

- Object-oriented SQL
- Works on **entity names, fields**, not on table/column names.
- Case-insensitive (except aliases).

Basic Example

java

 Copy code

```
Query q = session.createQuery("from Employee");
List<Employee> list = q.list();
```

Select specific fields

java

 Copy code

```
Query q = session.createQuery("select name, salary from Employee");
```

Where clause

java

 Copy code

```
Query q = session.createQuery("from Employee where salary > :sal");
q.setParameter("sal", 50000);
```

Update query

java

 Copy code

```
Query q = session.createQuery("update Employee set salary=60000 where id=1");
```

✓ Named Queries

These are predefined HQL queries stored at entity level.

Example:

java

Copy code

```
@Entity
@NamedQuery(
    name="findEmployeeByName",
    query="from Employee where name = :name"
)
public class Employee { ... }
```

Usage:

java

Copy code

```
Query q = session.getNamedQuery("findEmployeeByName");
q.setParameter("name", "Rahul");
List<Employee> list = q.list();
```

- ✓ Useful for reusable queries
- ✓ Good for performance

✓ Criteria Queries (Hibernate Criteria API)

Criteria API creates queries dynamically in **object-oriented manner** (no HQL string needed).

Basic Example:

java

Copy code

```
Criteria criteria = session.createCriteria(Employee.class);
List<Employee> employees = criteria.list();
```

Add Restrictions

java

Copy code

```
criteria.add(Restrictions.gt("salary", 50000));
```

Pagination

java

 Copy code

```
criteria.setFirstResult(0);  
criteria.setMaxResults(10);
```

Sorting

java

 Copy code

```
criteria.addOrder(Order.asc("name"));
```

Important Exam Questions

Short Notes / 2–5 Marks

1. What is Hibernate mapping?
2. Explain One-to-One mapping with annotation.
3. What is Component Mapping?
4. Write a note on `@ElementCollection`.
5. Difference between HQL and SQL.
6. What are Named Queries?
7. Explain lazy and eager loading.
8. What is CascadeType in Hibernate?

Long Answers / 10–15 Marks

1. Explain all Hibernate relationships with examples (One-to-One, One-to-Many, etc.).
2. What is Collection mapping and Component mapping? Explain with examples.
3. Compare HQL, Named Queries, and Criteria Queries with examples.
4. Explain Many-to-Many mapping with table structure and annotation code.

5. Explain Criteria API with Restrictions, Pagination, and Sorting examples.

If you want, I can also give:

- ✓ Diagrams for relationships
- ✓ Full mini-project example using Hibernate mappings
- ✓ Hibernate interview questions set

Just tell me!



Hibernate interview questions set

You're giving feedback on a new version of ChatGPT.

Which response do you prefer? Responses may take a moment to load.



Response 1

Here is a **complete, exam-oriented + interview-focused** **Hibernate Interview Question Set**.

These questions cover **basic → advanced**, perfect for **viva, technical interviews, and university exams**.



Hibernate Interview Questions (Most Important)

◆ Basic Level Questions

1. What is Hibernate?

Hibernate is an **ORM (Object Relational Mapping)** framework that maps **Java classes to database tables** and automates **CRUD operations**.



Response 2

Below is a **complete, exam-oriented** set of **Hibernate Interview Questions** categorized into **Basic, Intermediate, and Advanced** levels, with **model answers**—exactly how you should answer.



Hibernate Interview Questions (Most Important)

A. Basic Hibernate Interview Questions

1. What is Hibernate?

Hibernate is an **ORM (Object Relational Mapping)** framework that maps **Java classes to database tables** and eliminates **JDBC boilerplate code**.

2. What is ORM?

ORM (Object Relational Mapping) bridges the gap between **object-oriented Java** and **relational databases** by mapping classes ↔ tables, fields ↔ columns.

3. Why use Hibernate over JDBC?

- Reduces boilerplate code
- Automatically handles SQL
- Provides caching
- Independent of database
- Supports HQL, Criteria API
- Faster development

4. What is a SessionFactory in Hibernate?

- Thread-safe, heavyweight object
- Created once per application
- Reads hibernate.cfg.xml
- Creates **Session** objects

5. What is a Session?

- Lightweight, short-lived
- Represents a single unit of work
- Used to perform CRUD operations

6. What is a Transaction in Hibernate?

A transaction ensures **atomicity**.

Either **all operations succeed** or **none**.

7. What are persistent, transient, and detached states?

- **Transient**: Not associated with Session

2. What is ORM?

ORM is a technique that maps **object-oriented Java** to **relational database**.

3. What is the advantage of using Hibernate over JDBC?

- No SQL required (HQL)
- Automatic table mapping
- Caching → faster performance
- Less boilerplate code
- Database-independent

4. What is a Hibernate Session?

A Session is an interface that represents a **single unit of work** with the database used to **perform CRUD operations**.

5. What is SessionFactory?

It is a heavy-weight object that is used to obtain Session objects.

6. What is a Persistence Context?

It is the environment where the application manages entity states.

7. What are the different states of an entity?

- Transient
- Persistent
- Detached

- **Persistent:** Associated with Session
- **Detached:** Was persistent but Session is closed

◆ Intermediate Level Questions

8. What is Hibernate caching?

Caching improves performance by storing frequently accessed data.

Types:

1. **First Level Cache** (Session level) – default
2. **Second Level Cache** (SessionFactory level) – optional
3. **Query Cache** – stores HQL results

9. What is lazy loading in Hibernate?

Related entities are **loaded only when accessed**, not during initial fetch.

Annotation:

```
java
```

 Copy code

```
@OneToMany(fetch = FetchType.LAZY)
```

10. What are the inheritance mapping strategies?

Hibernate supports:

- **Single Table Strategy**
- **Table per Class (Concrete Class)**
- **Joined Strategy**

11. What are the Hibernate collection mappings?

- **List**

- **Removed**

8. What is HQL?

Hibernate Query Language that works on entity na

9. What is the difference between save() and persist()?

- **save()** returns generated ID
- **persist()** returns null

10. What is the difference between get() and load()?

- **get()** → **eager** loading
- **load()** → **lazy** loading, throws exception if not found

B. Intermediate Questions

11. What is lazy loading?

Hibernate loads related entities at the time of initial query.

12. What is cascade?

Cascade defines how changes to a child entity (persist, delete) affect the parent entity.

13. What are common Hibernate annotations?

- Set
- Map
- Bag
- Array

Using @ElementCollection

12. What are the types of primary key generation strategies?

- AUTO
- IDENTITY
- SEQUENCE
- TABLE
- UUID

Example:

java

 Copy code

```
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private int id;
```

13. What is HQL?

- Hibernate Query Language
- Works on **entity names**, not tables

14. What are Named Queries?

Pre-defined HQL queries written at entity level.

15. What is Criteria API?

Object-oriented way to build queries dynamically.

- CascadeType.ALL
- PERSIST
- REMOVE
- MERGE
- DETACH
- REFRESH

14. What is the difference between first-level and second-level cache?

- First-level cache -
- Second-level cache (optional: EhCache)

15. What is Criteria API?

API used to build dynamic HQL strings.

16. What is NamedQuery?

A predefined static HQL query. Annotated with @NamedQuery.

17. What are the inheritance strategies in Hibernate?

- Single Table
- Joined Table
- Table per Class

18. What is @Embeddable?

Used for **component** mapping inside entities (no separate table).

16. Difference: save() vs persist()

save()	persist()
Returns generated ID	Returns void
Can work outside transaction	Must be in transaction
Hibernate-specific	JPA-compliant

17. Difference: update() vs merge()

update()	merge()
Updates the same object	Copies changes to a new persistent object
Fails if object exists in session	Doesn't fail

◆ Advanced Level Questions

18. What is the N+1 Select Problem in Hibernate?

When fetching a parent entity, Hibernate executes **extra queries** for each child entity.

Fix:

- `fetch = FetchType.EAGER`
- Using JOIN FETCH in HQL
- Using Batch Fetching

19. What is Dirty Checking in Hibernate?

Hibernate automatically identifies changes in persistent objects and updates them without calling `update()` manually.

19. What is the purpose of the hbm2ddl file?

It contains:

- DB connection settings
- Hibernate properties
- Mapping files/classes

20. What is dirty checking?

Hibernate automatically identifies changes in persistent objects and updates them without calling `update()` manually.

21. What is flush() in Hibernate?

Synchronizes session state with the database.

22. What is evict() in Hibernate?

It removes an object from the session (changes its state to detached).

23. What is merge() in Hibernate?

Reattaches a detached object to the session.

24. What is the difference between update() and merge()?

- `update()` → fails if object is not in session
- `merge()` → works if object is not in session

25. What are orphaned objects?

20. What is the difference between First-level cache & Second-level cache?

First Level Cache	Second Level Cache
Session level	SessionFactory level
Mandatory	Optional
Disabled only by clearing Session	Requires external provider
Per user	Shared across users

Providers:

- EhCache
- EhCache 3
- Hazelcast
- Infinispan

21. What is the use of @Embeddable and @Embedded?

Used for **Component Mapping** (Value Objects inside an Entity).

22. What is JDBC batching in Hibernate?

Groups multiple insert/update statements into a single database round-trip.

23. What is the purpose of Hibernate.cfg.xml?

Contains:

- DB connection details
- Entity class mappings
- Hibernate properties

Deletes child entity when parent is deleted (orphanRemoval)

C. Advanced H Questions

26. What is N+1 select problem?

When fetching a parent entity, N child entities are fetched in N queries.

27. How to solve N+1 select problem?

- Use **fetch join** in HQL
- Use **batch-size**
- Use **second-level cache**

28. What is optimistic locking?

- **Optimistic locking** (uses version column)
- **Pessimistic locking** (uses lock update)

29. What is @Version?

Used for implementing optimistic locking.

30. What is FetchType.EAGER?

- **LAZY** → loads related entities on demand
- **EAGER** → loads related entities immediately

24. What is Query Cache in Hibernate?

Caches results of HQL/Criteria queries.

25. What are Hibernate interceptors?

Used to intercept CRUD operations to:

- Log events
- Audit
- Perform validation

26. What is Hibernate Validator?

Hibernate Validator uses **Bean Validation (JSR 380)** to apply constraints on fields:

Example:

```
java
```

 Copy code

```
@NotNull
@Size(min=3)
private String name;
```

27. What are orphanRemoval and Cascade?

Cascade Types

- PERSIST
- REMOVE
- MERGE
- REFRESH
- DETACH
- ALL

orphanRemoval

Automatically deletes child entities when removed from a collection.

31. What is the difference between openSession() and getSession()

- openSession() → returns a Session object
- getSession() → returns a Session object with a context

32. What is batch processing?

Executing multiple insert/update/delete operations using batching.

33. How does Hibernate manage sessions?

With strategies like:

- AUTO
- IDENTITY
- SEQUENCE
- TABLE

34. What is Session and Transaction?

- **Session:** connection to the database
- **Transaction:** group of operations executed atomically

35. What is the purpose of Hibernate?

Used for:

- Logging
- Validations
- Auditing

36. What is Hibernate?

28. What is the difference between ORM and JDBC?

ORM:

- No manual SQL
- Object-based
- Faster development

JDBC:

- Manual SQL required
- Verbose
- Developer handles everything manually

29. Can we use Hibernate without XML?

Yes, using only annotations.

30. What is @Entity and @Table annotation?

- @Entity → Marks class as Hibernate entity
- @Table → Defines table name

🌟 Bonus: Most Repeated Viva Questions

1. What is lazy loading?
2. Explain Hibernate architecture with diagram.
3. Explain entity states.
4. What is SessionFactory?
5. Explain inheritance mapping.
6. What is the use of @OneToMany?
7. What is HQL? How is it different from SQL?
8. Explain caching in Hibernate.
9. Difference between merge() and update().
10. What happens internally when session.save() is called?

Framework for adding annotations like:

- @NotNull
- @Size
- @Email

37. What are Hiberr

Tell Hibernate the SQL
MySQLDialect, OracleC

38. What is join fetch

Used to fetch associate

```
java
```

```
from Employee e jo:
```

39. What is SQL inje prevents it?

Hibernate uses:

- Parameter binding
 - HQL
 - Criteria API
- Which blocks SQL

40. How do you inte Spring?

Using:

- LocalSessionFact
- HibernateTemplat
- HibernateTransac

